

dataset_splitting_rules_engineering_ml

June 9, 2026

1 Dataset Splitting Rules for Machine Learning in Engineering Applications

1.1 Purpose

This note provides **general rules for splitting datasets** for machine-learning tasks, with an emphasis on **engineering use cases** where credibility, generalisation, and interpretability matter as much as raw predictive accuracy. It is intended to help teams design training, validation, and test splits that:

- reduce the risk of **data leakage**;
- improve the chance of **honest generalisation estimates**;
- support **interpretable model comparison**;
- align evaluation with the **intended engineering use case**.

This document is deliberately method-agnostic: the same principles apply to linear models, tree ensembles, Gaussian processes, neural networks, and hybrid / physics-guided methods.

1.2 Core principle: split to mimic the deployment question

The split strategy should reflect the real question you want the model to answer at deployment time.

Ask first:

- Will the model be used on **new rows from the same source and regime**?
- On **new experiments / facilities / campaigns / geometries**?
- On **future time periods**?
- On a **shifted operating regime**?
- On **rare but safety-relevant subgroups**?

A split is only useful if it tests the model under conditions that resemble the intended use case.

Engineering interpretation:

A random row split is often acceptable only when future data will truly be sampled independently from the same distribution and measurement process. In many engineering datasets, samples are clustered by source, facility, campaign, geometry, operating regime, or time. In those cases, a naive random split often overestimates performance.

1.3 Keep a strict train / validation / test separation

A good baseline workflow usually uses three logical roles:

- **Training set:** fit model parameters.
- **Validation set:** choose hyperparameters, compare models, tune preprocessing decisions.
- **Test set:** final performance check, used once at the end.

1.3.1 General rules

1. **Do not tune on the test set.**
2. **Do not repeatedly inspect the test set during development.**
3. If you use cross-validation inside model selection, keep a **final external holdout test set** whenever possible.

Why this matters:

Repeated use of the test set turns it into another validation set and inflates performance estimates.

1.4 Split before any data-dependent preprocessing

Any operation that learns from the data must be fit **using training data only**, then applied to validation/test data.

This includes, for example:

- scaling / normalisation;
- imputation;
- PCA or dimensionality reduction;
- feature selection;
- target encoding;
- resampling / balancing;
- outlier handling that uses whole-dataset statistics.

1.4.1 Safe rule

First split, then fit preprocessing on the training fold only.

1.4.2 Practical recommendation

Use a **pipeline** so the full preprocessing + model sequence is fit only inside the training data during validation or cross-validation.

Why this matters:

If preprocessing sees validation or test data, information leaks from the future evaluation set into model development.

1.5 Keep related samples together

If multiple rows come from the same underlying entity, they should usually be kept in the same split.

Typical engineering grouping variables include:

- experiment source;
- facility;
- campaign;
- geometry family;
- material batch;
- specimen;
- operating sequence;
- simulation case family;
- repeated measurements from the same condition.

1.5.1 Rule

If rows share source-specific bias or near-duplicate conditions, **group-split them together**.

1.5.2 Why this matters

A model may look excellent if it is tested on data that are only slightly different from rows already seen during training. This is one of the most common forms of hidden leakage in scientific and engineering datasets.

1.6 Match the split to the data structure

1.6.1 Independent, tabular, approximately IID data

Use a standard train/validation/test split or cross-validation, ideally with: - reproducible random seed; - stratification where appropriate (mainly for classification).

1.6.2 Grouped engineering data

If rows are grouped by source/facility/campaign, use: - **group holdout** for test; - **group-aware cross-validation** for validation; - explicit group labels stored with the dataset.

1.6.3 Time-ordered data

If the deployment target is future prediction, use: - **time-aware splitting**; - train on the past, validate/test on later periods; - never shuffle across time.

1.6.4 Regime-based engineering evaluation

If the deployment risk is tied to operating regime, add: - a **regime-balanced baseline split** for standard comparison; - and, where relevant, a **regime holdout / extrapolation split** to test performance outside the main training support.

1.7 Use a common baseline split for comparison across models

If several methods are being compared, they should share a **common organiser-owned baseline split** wherever possible.

This is especially important in a teaching or benchmarking context, because otherwise: - one method may appear better simply because it saw an easier split; - students may confuse split difficulty with model quality; - conclusions become hard to interpret.

1.7.1 Good practice

Prepare and distribute: - one master dataset; - one split-assignment file (e.g. `sample_id`, `split`); - optional convenience files for train/validation/test subsets.

This makes the comparison reproducible and transparent.

1.8 For engineering datasets, use regime-aware splitting when needed

In engineering, performance often varies by operating regime. A model that performs well on average may still fail badly in specific regions that matter for design or safety.

1.8.1 Practical approach

Define a few physically important variables, for example: - pressure; - mass flux; - temperature or subcooling; - quality; - geometry class.

Then use them to create **coarse regime labels** (for example low / medium / high bins), and check that train/validation/test each have reasonable regime coverage.

1.8.2 Important nuance

A regime-aware split does **not** mean every split must be perfectly identical. It means: - the **baseline** split should not accidentally omit important regimes; - the **special stress-test** split can deliberately hold out one regime to test extrapolation or safe-domain reasoning.

1.9 Separate ordinary testing from extrapolation / OOD testing

These are not the same thing.

1.9.1 Baseline test set

Should answer: > “How well does the model generalise to unseen but broadly in-domain data?”

1.9.2 Extrapolation / OOD / safe-domain holdout

Should answer: > “What happens when the model is applied outside its main support?”

For engineering practice, it is often useful to have **both**: 1. a baseline in-domain test set; and 2. a separate challenge set for regime shift, new sources, or out-of-domain use.

This is especially useful when teaching or assessing: - Gaussian processes; - uncertainty quantification; - decision support; - safe-use conditions.

1.10 Audit subgroup performance, not only overall averages

A single overall metric can hide important failures.

1.10.1 Always consider subgroup or regime-wise evaluation

Examples: - pressure bins; - mass-flux bins; - geometry families; - facilities or sources; - rare but safety-relevant cases.

1.10.2 Why this matters

A model can have strong average performance while failing systematically on an important subset. In engineering, those subsets may be precisely the cases that matter most.

1.11 Keep the split logic documented and reproducible

Every project should record:

- which rows belong to train / validation / test;
- the grouping variable(s), if any;
- the rationale for the split;
- any special holdout sets;
- the random seed (if used);
- the intended deployment question the split is meant to approximate.

1.11.1 Minimum recommendation

Store the split as a file, not only as notebook code.

Example:

```
sample_id,source_id,split
1,S12,train
2,S12,train
3,S41,val
4,S07,test
```

This is far easier to audit and reproduce than re-running ad hoc splitting code.

1.12 Recommended practical rules (short version)

1.12.1 Always do

- Split before fitting preprocessing.
- Keep train / validation / test logically separate.

- Use grouped splits when samples are related by source, facility, subject, campaign, or geometry.
- Use time-aware splits for forecasting or future deployment.
- Use a common baseline split for model comparison.
- Evaluate subgroup / regime performance, not just overall averages.
- Keep an external holdout test set whenever feasible.
- Document and save the split assignments.

1.12.2 Avoid

- Random row splitting when rows are grouped or correlated.
 - Fitting scalars / imputers / PCA on the full dataset before splitting.
 - Re-using the test set repeatedly during model development.
 - Comparing methods across different splits without clear justification.
 - Claiming “generalisation” when evaluation only tested interpolation within a narrowly shared source.
-

1.13 Engineering-specific interpretation

For engineering usage, the split design should support **credibility**, not just convenience.

In practice, this means:

- the model should be tested on data that reflect its **intended operating conditions**;
- uncertainty and safe-use claims should be based on **validation and holdout evidence**, not only on training metrics;
- extrapolation claims should be treated with caution unless the split explicitly evaluates them;
- if the model may influence design, operations, or safety decisions, the evaluation should align with broader **verification, validation, and uncertainty-quantification (VVUQ / TEVV)** thinking.

In short:

A good split is part of the model’s engineering credibility argument.

1.14 A simple decision guide

1.14.1 Use a random split when:

- samples are approximately independent;
- there is no grouping or time ordering;
- deployment will be on similar future samples from the same distribution.

1.14.2 Use a grouped split when:

- rows come from the same source / campaign / facility / geometry / subject;
- repeated or correlated measurements exist;
- you want to test generalisation to new groups.

1.14.3 Use a time-based split when:

- the model will predict the future from the past;
- time leakage is possible;
- the data generating process may drift over time.

1.14.4 Add a regime / OOD holdout when:

- operating regime matters;
 - deployment may encounter shifted conditions;
 - safe-domain reasoning is important;
 - uncertainty-aware decision support is part of the use case.
-

1.15 Suggested workflow for an engineering ML project

1. Define the **deployment question**.
 2. Identify possible sources of leakage:
 - grouping;
 - repeated measurements;
 - future information;
 - preprocessing on all data.
 3. Choose a **baseline split** that reflects standard use.
 4. Add **special holdouts** if you also need extrapolation / OOD assessment.
 5. Build preprocessing inside a pipeline.
 6. Tune only on training/validation.
 7. Use the final test set once.
 8. Report both overall and subgroup/regime performance.
 9. Save the split assignments and document the rationale.
-

1.16 References

1. **scikit-learn — Common pitfalls and recommended practices**
https://scikit-learn.org/stable/common_pitfalls.html
Useful for the general rule that preprocessing must be fit on training data only and for scikit-learn's formal description of data leakage.
2. **scikit-learn — Cross-validation: evaluating estimator performance**
https://scikit-learn.org/stable/modules/cross_validation.html
Useful for the general logic of validation and model evaluation, including grouped cross-validation.
3. **scikit-learn — GroupKFold**
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupKFold.html
Useful for grouped datasets where the same group must not appear in both train and test.
4. **scikit-learn — TimeSeriesSplit**
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

Useful for time-ordered datasets where training on the future and testing on the past would be invalid.

5. **Kaufman, Rosset, Perlich, Stitelman (2012), “Leakage in Data Mining: Formulation, Detection, and Avoidance”**
https://www.cs.umb.edu/~ding/history/470_670_fall_2011/papers/cs670_Tran_PreferredPaper_Leaking
A classic reference on what leakage is and why it leads to over-optimistic estimates.
6. **NIST — AI Test, Evaluation, Validation and Verification (TEVV)**
<https://www.nist.gov/ai-test-evaluation-validation-and-verification-tevv>
Useful for the broader point that trustworthy AI depends on reliable evaluation and measurement.
7. **ASME — VVUQ information**
<https://www.asme.org/codes-standards/publications-information/verification-validation-uncertainty>
Useful for the engineering perspective that evaluation should support credibility, verification, validation, and uncertainty-aware use.
8. **Yeo, Romeijn, et al. (2020), “A Summary of Industrial Verification, Validation, and Uncertainty Quantification Procedures” (NIST)**
<https://www.nist.gov/publications/summary-industrial-verification-validation-and-uncertainty-quantification-procedures>
Useful for engineering-oriented framing of validation, uncertainty, and credibility.
9. **Oakden-Rayner et al. (2020), “Hidden Stratification Causes Clinically Meaningful Failures in Machine Learning for Medical Imaging”**
<https://pmc.ncbi.nlm.nih.gov/articles/PMC7665161/>
Although from medical imaging, this is a strong reference for the general point that good average performance can hide serious subgroup failures, which is also highly relevant to engineering regime-wise evaluation.
10. **Dockès et al. (2021), “Preventing dataset shift from breaking machine-learning biomarkers”**
<https://pmc.ncbi.nlm.nih.gov/articles/PMC8478611/>
Useful for the general problem of dataset shift and the need to evaluate beyond naive in-sample assumptions.

1.17 Closing remark

For engineering ML, **dataset splitting is not just a technical detail**. It is one of the main controls that determine whether a performance estimate is trustworthy, whether a model comparison is fair, and whether the resulting model can be interpreted responsibly in practice.